

Namo

Named dimensional data for Ruby.

Namo is a Ruby library for working with multi-dimensional data using named dimensions. It infers dimensions and coordinates from plain arrays of hashes — the same shape you get from databases, CSV files, JSON, and YAML — so there's no reshaping step.

The design rests on a few stances: every hash key is a dimension and none is privileged as a coordinate or value; formulae attach to a Namu alongside data and re-evaluate on each access, appearing as derived dimensions alongside the data dimensions; operators that combine Namos all take Namos and return Namos, so analytical pipelines close; and the formula mechanism is type-agnostic — strings, dates, booleans, and arbitrary Ruby objects work as readily as numbers.

Installation

```
gem install namo
```

Or in your Gemfile:

```
gem 'namo'
```

Usage

Create a Namu instance from an array of hashes:

```
require 'namo'

sales = Namu.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])
```

Dimensions and coordinates are inferred:

```
sales.dimensions
# => [:product, :quarter, :price, :quantity]
```

```
sales.coordinates[:product]
# => ['Widget', 'Gadget']
```

```
sales.coordinates[:quarter]
# => ['Q1', 'Q2']
```

Every key is a dimension; every value is a coordinate. There's no schema declaration and no choosing which column is “the index” — price and quantity are no less first-class than product and quarter.

Selection

Select by named dimension using keyword arguments:

```
# Single value
sales[product: 'Widget']
# => #<Namu [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>
```

```

# ]>

# Multiple dimensions
sales[product: 'Widget', quarter: 'Q1']
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100}
# ]>

# Range
sales[price: 10.0..20.0]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

# Array of values
sales[quarter: ['Q1']]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
# ]>

# Proc predicate
sales[price: ->(v){v < 20.0}]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

# Regex predicate
sales[product: /^W/]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

```

Procs receive the dimension value and select the row when they return truthy. They handle arbitrary predicates — multi-condition tests, nil-aware checks, anything Ruby can express — and compose with everything else:

```

sales[price: ->(v){v < 20.0}, quantity: ->(v){v > 100}]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>

```

Regexes match against the dimension value coerced with `to_s`, so they work against strings, symbols, numbers, dates, or anything else with a sensible string form. `nil` becomes `""` — `//` matches it, `./.` doesn't.

```

sales[product: /widget/i]           # case-insensitive
sales[product: /Widget|Gadget/]     # alternation
sales[product: /^W/, quarter: 'Q1'] # mixed with exact

```

Procs and regexes mix freely with exact values, arrays, ranges, projection, and contraction in the same `[]` call.

Projection

Project to specific dimensions:

```
sales[:product, :price]
# => #<Namo [
#   {product: 'Widget', price: 10.0},
#   {product: 'Widget', price: 10.0},
#   {product: 'Gadget', price: 25.0},
#   {product: 'Gadget', price: 25.0}
# ]>
```

Selection and projection can be chained:

```
sales[product: 'Widget'][:quarter, :price]
# => #<Namo [
#   {quarter: 'Q1', price: 10.0},
#   {quarter: 'Q2', price: 10.0}
# ]>
```

Or combined in a single call (names before selectors):

```
sales[:quarter, :price, product: 'Widget']
# => #<Namo [
#   {quarter: 'Q1', price: 10.0},
#   {quarter: 'Q2', price: 10.0}
# ]>
```

Contraction

Contraction is the complement of projection. Projection says “keep these dimensions”; contraction says “remove these dimensions, keep everything else”:

```
sales[-:price, -:quantity]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1'},
#   {product: 'Widget', quarter: 'Q2'},
#   {product: 'Gadget', quarter: 'Q1'},
#   {product: 'Gadget', quarter: 'Q2'}
# ]>
```

The `-:price` syntax uses unary minus on Symbol to produce a negated dimension. Mixing projection and contraction in the same call is an error — the two modes are mutually exclusive:

```
sales[:product, -:price] # => ArgumentError
```

Selection and contraction can be chained:

```
sales[product: 'Widget'][-:price, -:quantity]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1'},
#   {product: 'Widget', quarter: 'Q2'}
# ]>
```

Or combined in a single call (names before selectors):

```
sales[-:price, -:quantity, product: 'Widget']
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1'},
#   {product: 'Widget', quarter: 'Q2'}
# ]>
```

```
# ]>
```

Selection, projection, and contraction always return a new Namo instance, so everything chains.

Concatenation

+ is the first of Namo's binary operators: it takes a Namo on each side and returns a Namo. The same shape holds for -, &, |, ^, ==, ===, <, <=, >, >= and (later) the composition operators — Namo in, Namo (or boolean) out — so analytical pipelines stay queryable end-to-end.

+ combines two Namo objects that share the same dimensions by appending the rows of the second to the first:

```
q1_sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

q2_sales = Namo.new([
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

all_sales = q1_sales + q2_sales
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
#   {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
# ]>
```

The dimensions must match — concatenating Namo objects with different dimensions raises an ArgumentError. Formulae carry through from the left-hand side.

Row Removal

- removes from the first Namo any row that appears exactly in the second:

```
sales = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

discontinued = Namo.new([
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

sales - discontinued
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150}
# ]>
```

Removal is exact — every dimension, every value must match. The dimensions must match; different dimensions raise an `ArgumentError`. Formulae carry through from the left-hand side.

Intersection

`&` returns the rows present in both `Namo` objects, like `Array#&`:

```
sales = Namu.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Widget', quarter: 'Q2', price: 10.0, quantity: 150},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

confirmed = Namu.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
])

sales & confirmed
# => #<Namu [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q2', price: 25.0, quantity: 60}
# ]>
```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae carry through from the left-hand side.

Union

`|` returns all rows from both sides, deduplicated, like `Array#|`:

```
q1_sales = Namu.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

all_sales = Namu.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Thingy', quarter: 'Q3', price: 5.0, quantity: 10}
])

q1_sales | all_sales
# => #<Namu [
#   {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Thingy', quarter: 'Q3', price: 5.0, quantity: 10}
# ]>
```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae merge from both sides; the left-hand side's formulae take precedence on conflict.

Symmetric Difference

`^` returns rows that appear in one side but not both:

```

set_a = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40}
])

set_b = Namo.new([
  {product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100},
  {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
])

set_a ^ set_b
# => #<Namo [
#   {product: 'Gadget', quarter: 'Q1', price: 25.0, quantity: 40},
#   {product: 'Thingo', quarter: 'Q3', price: 5.0, quantity: 10}
# ]>

```

The dimensions must match; different dimensions raise an `ArgumentError`. Formulae merge from both sides; the left-hand side's formulae take precedence on conflict.

Composition

`*` is the equi-join operator. It pairs rows from two Namos where coordinates match on every shared dimension, like an inner join on the shared dimension names:

```

ohlcv = Namo.new([
  {symbol: 'BHP', date: '2025-01-01', close: 42.5},
  {symbol: 'RIO', date: '2025-01-01', close: 118.3}
])

fundamentals = Namo.new([
  {symbol: 'BHP', pe: 14.5},
  {symbol: 'RIO', pe: 9.2}
])

ohlcv * fundamentals
# => #<Namo [
#   {symbol: 'BHP', date: '2025-01-01', close: 42.5, pe: 14.5},
#   {symbol: 'RIO', date: '2025-01-01', close: 118.3, pe: 9.2}
# ]>

```

Inner-join semantics: unmatched rows from either side are dropped. Output dimensions are `self.data_dimensions` followed by `other.data_dimensions` exclusive to other. Duplicates on shared coordinates are preserved multiplicatively — output multiplicity is the product of input multiplicities on each matching key.

The two Namos must have at least one shared data dimension. No overlap raises an `ArgumentError` — the asymmetry with `**` is deliberate, and falling through to a Cartesian product would silently turn a logic error into a large pile of nonsense rows. Formulae merge from both sides; the left-hand side wins on conflict.

Cartesian product

`**` is the Cartesian product. Every row from the left paired with every row from the right:

```

products = Namo.new([product: 'Widget'], {product: 'Gadget'})
quarters = Namo.new([quarter: 'Q1'], {quarter: 'Q2'})

products ** quarters
# => #<Namo [

```

```
# {product: 'Widget', quarter: 'Q1'},
# {product: 'Widget', quarter: 'Q2'},
# {product: 'Gadget', quarter: 'Q1'},
# {product: 'Gadget', quarter: 'Q2'}
# ]>
```

Output has `self.data.length * other.data.length` rows. Output dimensions are `self.data_dimensions + other.data_dimensions`, in operand order. Duplicates are preserved multiplicatively.

The two Namos must have **no** shared data dimensions — the precondition is the mirror image of `*`. Any overlap raises an `ArgumentError`; allowing it would produce rows with the same dimension named twice. Formulae merge from both sides; the left-hand side wins on conflict.

The visual relationship is intentional: `*` is the filtered version, `**` is the explosive version — more sigil, more output.

Decomposition

`/` removes from the left Namo the dimensions that are also in the right, then dedupes the projected rows. It's the inverse of `*` and `**`:

```
combined = Namo.new([
  {symbol: 'BHP', date: '2025-01-01', close: 42.5, pe: 14.5},
  {symbol: 'RIO', date: '2025-01-01', close: 118.3, pe: 9.2}
])
```

```
fundamentals = Namo.new([
  {symbol: 'BHP', pe: 14.5},
  {symbol: 'RIO', pe: 9.2}
])
```

```
combined / fundamentals
# => #<Namo [
#   {date: '2025-01-01', close: 42.5},
#   {date: '2025-01-01', close: 118.3}
# ]>
```

The intersection of dimensions — here `:symbol` and `:pe` — is removed. Everything else stays. The projected rows are deduplicated, so `/` answers “what’s left when these dimensions are factored out?” rather than “what rows survive a column drop?”. Formulae carry through from the left-hand side.

`/` has no precondition. When the two Namos share no dimensions, the intersection is empty, nothing is removed, and `self / other` returns a Namo equal to `self`:

```
shipments = Namo.new([{:order_id: 1, weight: 10}])
weather = Namo.new([{:date: '2025-01-01', temperature: 22}])
```

```
shipments / weather
# => #<Namo [{:order_id: 1, weight: 10}]> – equal to shipments
```

The round-trip identity holds for the `**` case exactly:

```
a = Namo.new([{:symbol: 'BHP'}, {:symbol: 'RIO'}])
b = Namo.new([{:quarter: 'Q1'}, {:quarter: 'Q2'}])
```

```
(a ** b) / b == a
# => true
```

For `*`, the round-trip is lossy on the dimensions that were shared between the operands:

```
a = Namo.new([{:symbol: 'BHP', close: 42.5}, {:symbol: 'RIO', close: 118.3}])
b = Namo.new([{:symbol: 'BHP', pe: 14.5}, {:symbol: 'RIO', pe: 9.2}])
```

```
(a * b) / b
# => #<Namo [{:close: 42.5}, {:close: 118.3}]>
# Equal to a[-:symbol]. :symbol was shared and is lost.
```

The asymmetry is inherent: `/` operates only on the two values it receives and can't distinguish “shared dimension that belonged to both” from “exclusive dimension that belonged only to the right”. Removing the intersection is the only rule expressible from the operands alone, and it gives clean recovery from `**` and well-defined (if lossy) recovery from `*`.

Why `/` is loose `*` and `**` raise when their preconditions are violated — combining unrelated Namos has no natural answer, and silently producing arbitrary output would turn a logic error into a large pile of nonsense rows. `/` is different: it's a projecting operator, not a combining one, and projecting away nothing returns the original. The no-precondition rule isn't a fallback; it's the structurally correct result.

This earns `/` three properties a strict version would lose:

- **Identity test.** `combined / other == combined` exactly when the two have no shared dimensions — answers “are these Namos dimensionally independent?” without explicit introspection. Same shape as `a & b == a` answering subset from 0.6.0.
- **Idempotence.** `(c / b) / b == c / b`. Once `b`'s dimensions are removed, removing them again does nothing.
- **Pipeline composition.** A processing step that applies `/ separator` can run over any Namo regardless of whether the separator's dimensions apply. Uninvolved Namos pass through unchanged; involved Namos get stripped. The pipeline doesn't need to special-case applicability.

This is the same pattern that makes `Array#-` useful with arrays that aren't subsets: `[1, 2, 3] - [9] == [1, 2, 3]`, not an error. The no-op-on-non-applicable behaviour lets the operator compose into pipelines that don't know in advance whether the operation applies.

Equality

Comparison on Namos is **multiset-theoretic on rows**: row order is ignored (it's an accident of ingestion, not data), but row multiplicities count (they *are* data). The same stance carries across the equality, pattern-match, and subset/superset operators below.

`==` is multiset equality on rows. Class and formulae are ignored; row order is ignored; row multiplicities are not.

```
a = Namo.new([{:x: 1}, {:x: 2}])
b = Namo.new([{:x: 2}, {:x: 1}])
```

```
a == b
# => true
```

```
a == Namo.new([{:x: 1}, {:x: 1}, {:x: 2}])
# => false
```

`eql?` is stricter: it also requires the class to match and the formula names to match. Like `===`, it ignores proc bodies — proc identity isn't a meaningful equivalence in Ruby (`proc{...} == proc{...}` is false), so neither `===` nor `eql?` uses it.

`hash` is consistent with `eql?` and is content-based, so equal Namos hash equally and can be used as Hash keys:

```
h = {a => 'first'}
```



```
h[b]
# => 'first'
```

`equal?` is unchanged from Ruby's default — it tests object identity.

`===` answers a different question: does the candidate have the same dimensions and the same formula names? Row data is ignored, and so are the proc bodies themselves — only the names matter. This is the `===` semantics that case statements use, so Namos can serve as templates for analytical shape:

```
sales_shape = Namo.new([{:product: 'X', quarter: 'Q1', price: 0.0, quantity: 0}])
sales_shape[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

```
q1 = Namo.new([{:product: 'Widget', quarter: 'Q1', price: 10.0, quantity: 100}])
q1[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

```
sales_shape === q1
# => true (same dimensions, same formula name)
```

```
sales_shape == q1
# => false (different rows)
```

The two `:revenue` procs are independently-written and not the same object — `proc{...} == proc{...}` is false in Ruby. But `===` doesn't compare proc identity; it asks “do these Namos have the same analytical shape?” and the shape is the set of dimensions plus the set of formula names.

Each comparison operator answers a distinct question: `eq?` is strictest (class + data + formula names); `==` is data identity; `===` is analytical identity; the subset operators are data containment.

Subset and Superset

`<`, `<=`, `>`, `>=` are multiset subset and superset relations on rows.

```
small = Namo.new([{:x: 1}, {:x: 2}])
large = Namo.new([{:x: 1}, {:x: 2}, {:x: 3}])
```

```
small <= large
# => true
```

```
small < large
# => true
```

```
large > small
# => true
```

Equal sets are `<=` and `>=` each other, but neither `<` nor `>`. Disjoint sets are none of the above — unless one side is empty, in which case it is a subset of (and disjoint with) the other.

Multiplicity matters: a single `{x: 1}` is a proper subset of two `{x: 1}`s.

```
one = Namo.new([{:x: 1}])
two = Namo.new([{:x: 1}, {:x: 1}])
```

```
one < two
# => true
```

The dimensions must match; different dimensions raise an `ArgumentError`. Comparing against a non-Namo raises a `TypeError`.

Formulae

Define computed dimensions using []=:

```
sales[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

```
sales[:product, :quarter, :revenue]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Widget', quarter: 'Q2', revenue: 1500.0},
#   {product: 'Gadget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Gadget', quarter: 'Q2', revenue: 1500.0}
# ]>
```

Formulae aren't materialised into row data — they re-evaluate on every access. A `:revenue` value reflects the current `:price` and `:quantity` at the moment you ask for it, so derived values stay in sync with whatever the underlying data is doing.

Formulae compose:

```
sales[:cost] = proc{|row| row[:quantity] * 4.0}
sales[:profit] = proc{|row| row[:revenue] - row[:cost]}
```

```
sales[:product, :quarter, :profit]
# => #<Namo [
#   {product: 'Widget', quarter: 'Q1', profit: 600.0},
#   {product: 'Widget', quarter: 'Q2', profit: 900.0},
#   {product: 'Gadget', quarter: 'Q1', profit: 840.0},
#   {product: 'Gadget', quarter: 'Q2', profit: 1260.0}
# ]>
```

Formulae work with selection and projection:

```
sales[product: 'Widget'][:revenue, :quarter]
# => #<Namo [
#   {revenue: 1000.0, quarter: 'Q1'},
#   {revenue: 1500.0, quarter: 'Q2'}
# ]>
```

Formulae carry through selection — a filtered Namo instance remembers its formulae.

Coordinates and values

`dimensions` covers the *queryable namespace* — every name you can ask for, whether it lives in the row data or is computed by a formula. Once formulae are defined, they appear alongside data dimensions:

```
sales[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

```
sales.dimensions
# => [:product, :quarter, :price, :quantity, :revenue]
```

```
sales.data_dimensions
# => [:product, :quarter, :price, :quantity]
```

```
sales.derived_dimensions
# => [:revenue]
```

`coordinates` gives the unique values per dimension, including derived ones:

```
sales.coordinates[:product]
# => ['Widget', 'Gadget']
```

```
sales.coordinates[:revenue]
# => [1000.0, 1500.0]
```

values gives the full per-row sequence — duplicates preserved, row order preserved:

```
sales.values[:product]
# => ['Widget', 'Widget', 'Gadget', 'Gadget']
```

```
sales.values[:revenue]
# => [1000.0, 1500.0, 1000.0, 1500.0]
```

Both coordinates and values accept positional arguments. With no args they return a Hash across the queryable namespace; with one arg they lazily compute and return just that column as an Array; with multiple args they return a subset Hash containing just the requested columns:

```
sales.values(:product)
# => ['Widget', 'Widget', 'Gadget', 'Gadget']
```

```
sales.values(:product, :quarter)
# => {
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2']
# }
```

```
sales.coordinates(:revenue)
# => [1000.0, 1500.0]
```

Single-arg access is lazy: sales.values(:revenue) evaluates the formula only across the rows of :revenue, without materialising the other columns. The bracket form (sales.values[:revenue]) still works through ordinary Hash lookup but pays for the full materialisation up front.

coordinates is values with .uniq applied per column — coordinates(dim) == values(dim).uniq holds for every dimension.

to_h is the Ruby-conventional alias for the full values Hash:

```
sales.to_h
# => {
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2'],
#   price: [10.0, 10.0, 25.0, 25.0],
#   quantity: [100, 150, 40, 60],
#   revenue: [1000.0, 1500.0, 1000.0, 1500.0]
# }
```

Unknown dimensions propagate nil per row — values(:missing) returns [nil, nil, ...] rather than raising or returning a sentinel, matching the convention used by Row#[] and [] selection. Use dimensions.include?(:dim) if you need to check membership directly.

Enumerable

Namo includes Enumerable, so each, reduce, map, select, min_by, and all the rest work out of the box. Rows are yielded as Row objects, so formulae are accessible during enumeration:

```
sales.reduce(0){|sum, row| sum + row[:quantity]}
# => 350
```

```

sales[product: 'Widget'].reduce(0){|sum, row| sum + row[:quantity]}
# => 250

sales[:revenue] = proc{|row| row[:price] * row[:quantity]}

sales.reduce(0){|sum, row| sum + row[:revenue]}
# => 5000.0

sales[product: 'Widget'].reduce(0){|sum, row| sum + row[:revenue]}
# => 2500.0

sales.map{|row| row[:product]}
# => ['Widget', 'Widget', 'Gadget', 'Gadget']

sales.min_by{|row| row[:price]}[:product]
# => 'Widget'

sales.flat_map{|row| [row[:price]]}
# => [10.0, 10.0, 25.0, 25.0]

```

Extracting data

`to_a` returns an array of hashes — the row-oriented form:

```

sales[:product, :quarter, :revenue].to_a
# => [
#   {product: 'Widget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Widget', quarter: 'Q2', revenue: 1500.0},
#   {product: 'Gadget', quarter: 'Q1', revenue: 1000.0},
#   {product: 'Gadget', quarter: 'Q2', revenue: 1500.0}
# ]

```

`to_h` returns a hash of arrays — the columnar form (see Coordinates and values above):

```

sales[:product, :quarter, :revenue].to_h
# => {
#   product: ['Widget', 'Widget', 'Gadget', 'Gadget'],
#   quarter: ['Q1', 'Q2', 'Q1', 'Q2'],
#   revenue: [1000.0, 1500.0, 1000.0, 1500.0]
# }

```

Why?

Every other multi-dimensional array library requires you to pre-shape your data before you can work with it. Namo takes it in the form it likely already comes in.

Name

Namo: nam(ed) (dimensi)o(ns). A companion to Numo (numeric arrays for Ruby). And in Aussie culture ‘o’ gets added to the end of names.

Contributing

1. Fork it (<https://github.com/thoran/namo/fork>)

2. Create your feature branch (git checkout -b my-new-feature)
3. Commit your changes (git commit -am 'Add some feature')
4. Push to the branch (git push origin my-new-feature)
5. Create a new pull request

License

MIT